

COMPILER DESIGN PROJECT

Software Requirements Specification

Code Summarization Using AST and NLP

Week 3 Deliverable

Submitted to: Course Instructor

Course: Compiler Design

Date: February 2026

Version 1.0

Contents

1	Introduction	4
1.1	Purpose	4
1.2	Scope	4
1.3	Definitions, Acronyms, and Abbreviations	4
2	Non-Functional Requirements	4
2.0.1	Usability Requirements	4
3	Appendices	4
3.1	Appendix C: Document Revision History	4

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for an automated code summarization tool designed for security audits and code reviews. The system leverages Abstract Syntax Tree (AST) analysis and Natural Language Processing (NLP) techniques to generate human-readable summaries of Python source code.

The intended audience for this document includes:

- Project developers and implementers
- Security auditors who will use the tool
- Academic reviewers and course instructors
- Future maintainers of the codebase

1.2 Scope

The **AST-NLP Code Summarizer** is a command-line tool that analyzes Python source code to produce structured summaries for security audit purposes.

The system will:

- Parse Python files into Abstract Syntax Trees without executing code
- Extract structural information (functions, classes, imports, control flow)
- Generate summaries using both template-based and ML-based approaches
- Detect common security vulnerabilities and anti-patterns
- Support batch processing of entire codebases
- Produce human-readable reports for auditors

Out of Scope

- Analysis of non-Python languages
- Dynamic analysis or code execution
- Automated vulnerability patching
- Integration with IDEs or GUI interfaces
- Real-time code analysis during development

1.3 Definitions, Acronyms, and Abbreviations

2 Non-Functional Requirements

2.0.1 Usability Requirements

3 Appendices

3.1 Appendix C: Document Revision History

Term	Definition
AST	Abstract Syntax Tree - a tree representation of source code structure
NLP	Natural Language Processing - AI techniques for understanding human language
ML	Machine Learning - algorithms that improve through experience
CLI	Command-Line Interface - text-based user interface
CodeT5	Pre-trained transformer model for code understanding tasks
Template-based	Rule-based summarization using predefined patterns
Security Audit	Systematic evaluation of code for vulnerabilities
Static Analysis	Code analysis without execution
SRS	Software Requirements Specification
FR	Functional Requirement
NFR	Non-Functional Requirement
SR	Security Requirement

Table 1: Terminology

ID	Requirement
NFR11	The system shall use clear, non-technical language in error messages
NFR12	The system shall provide numbered menu options for easy selection
NFR13	The system shall use visual indicators (✓ / X / !) to indicate status
NFR14	The system shall display progress for long-running operations
NFR15	The system shall confirm destructive actions

Table 2: Usability Requirements

Version	Date	Author	Changes
1.0	Feb 2026	Project Team	Initial release

Table 3: Revision History